

INTRODUCTION TO JAVASCRIPT

1. Introduction to JavaScript

JavaScript is a programming language mainly used to add **interactivity and dynamic behavior to web applications**. It was created by **Brendan Eich** and was initially developed to provide simple interactivity in the Netscape browser. Today, JavaScript is an important technology along with **HTML and CSS** for web application development.

Where JavaScript Runs

JavaScript can run in different environments:

- **Web Browser:** Used for client-side scripting, user interaction, and DOM manipulation.
- **Node.js:** Used for server-side scripting, backend applications, and APIs.
- **Cross-platform environments:** JavaScript can also be used for desktop, mobile, and IoT applications with appropriate frameworks.

HTML, CSS and JavaScript

A simple way to understand their roles is:

Technology Purpose

HTML Provides the structure of a webpage

CSS Provides styling and layout

JavaScript Provides behavior and interactivity

For example, HTML can create a button, CSS can style it, and JavaScript can define what happens when the button is clicked.

2. Basics of JavaScript

2.1 Variables

Variables are used to store data in a JavaScript program.

JavaScript provides three main ways to declare variables:

var

- Used in older JavaScript code.
- Was commonly used before 2015.

let

- Introduced in ES6/2015.
- A variable declared using let cannot be redeclared in the same scope.

const

- Introduced in ES6/2015.
- Cannot be redeclared or reassigned.
- Has block scope.

Recommended Usage

According to the training material:

1. Always declare variables.
2. Use const when the value should not change.
3. Use const when the variable's type should not change, including arrays and objects.
4. Use let when const cannot be used.
5. Use var mainly when old-browser support is required.

Example:

```
const name = "Divya";
```

```
let age = 20;
```

```
age = 21;
```

Here, name cannot be reassigned, while age can be changed.

3. JavaScript Data Types

JavaScript supports different types of data.

String

Used to represent text.

```
let name = "ApexaiQ";
```

Number

Used for integer and decimal values.

```
let age = 25;
```

```
let price = 3.14;
```

Boolean

Represents either true or false.

```
let isLoggedIn = true;
```

Null

Represents an intentionally empty value.

```
let data = null;
```

Undefined

Occurs when a variable has been declared but has not been assigned a value.

```
let result;
```

Object

Stores data in **key-value pairs**.

```
let student = {  
  name: "Divya",  
  age: 20
```

```
};
```

Array

Stores an ordered collection of values.

```
let subjects = ["Java", "Python", "JavaScript"];
```

The training document identifies String, Number, Boolean, Null, Undefined, Object, and Arrays as important JavaScript data types.

4. JavaScript Operators

Operators are symbols used to perform operations on values.

Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Comparison Operators

==	Equal
===	Strictly equal
!=	Not equal
<	Less than
>	Greater than

Logical Operators

&&	AND
	OR
!	NOT

Assignment Operators

=	Assignment
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign

These operators are included in the JavaScript training material as fundamental programming concepts.

5. Control Flow

Control flow determines **which statements are executed and how many times they are executed.**

5.1 Conditional Statements

if

Executes a block of code when a specified condition is true.

```
if (age >= 18) {  
    console.log("Eligible");  
}
```

else

Executes when the if condition is false.

```
if (age >= 18) {  
    console.log("Eligible");  
} else {  
    console.log("Not Eligible");  
}
```

else if

Used when multiple conditions need to be checked.

```
if (marks >= 75) {  
    console.log("Distinction");  
} else if (marks >= 50) {  
    console.log("Pass");  
} else {  
    console.log("Fail");  
}
```

switch

Used when there are multiple alternative cases.

```
switch (day) {  
    case 1:  
        console.log("Monday");  
        break;  
    case 2:  
        console.log("Tuesday");  
        break;  
    default:  
        console.log("Invalid");  
}
```

5.2 Loops

Loops execute a block of code repeatedly.

- **for:** Repeats code a specified number of times.

- **for/in:** Iterates through properties of an object.
- **for/of:** Iterates through values of an iterable.
- **while:** Executes while a condition is true.
- **do/while:** Executes the block first and then checks the condition.

6. Functions

A function is a **block of code designed to perform a specific task**. A function executes when it is called or invoked and helps in **code reusability**.

Important Components

- **Parameters:** Input values given to a function.
- **Return:** Output produced by a function.
- **Scope:** Determines where variables can be accessed.

Types of Scope

- Global scope
- Local scope
- Block scope

Example

```
function add(a, b) {
    return a + b;
}
```

```
let result = add(10, 20);
console.log(result);
```

7. Types of Functions

The training material covers several types of functions.

Function Declaration

A named function defined using the function keyword.

```
function greet() {
    console.log("Hello");
}
```

Function declarations are **hoisted**, meaning they can be called before their declaration.

Function Expression

A function stored inside a variable.

```
const greet = function() {
    console.log("Hello");
}
```

```
};
```

Arrow Function

Uses the `=>` syntax and provides shorter function syntax.

```
const add = (a, b) => a + b;
```

Arrow functions do not have their own `this`.

Anonymous Function

A function without a name, commonly used as a callback.

IIFE

Immediately Invoked Function Expression executes automatically after being defined.

Higher-Order Function

A function that either:

Takes another function as an argument, or

Returns another function.

Recursive Function

A function that calls itself.

8. Arrays

An array is an **ordered collection of values**. It can contain numbers, strings, objects, and even functions.

Example:

```
let languages = ["Java", "Python", "JavaScript"];
```

Important Array Properties and Methods

Method	Purpose
<code>.length</code>	Returns total number of items
<code>.push()</code>	Adds an item at the end
<code>.pop()</code>	Removes an item from the end
<code>.shift()</code>	Removes the first item
<code>.unshift()</code>	Adds an item at the beginning
<code>.map()</code>	Processes array elements
<code>.filter()</code>	Filters elements based on a condition
<code>.reduce()</code>	Reduces array values into a result

9. Objects

An object is a **collection of key-value pairs**.

Example:

```
let student = {  
    name: "Divya",  
    age: 20,  
    course: "Computer Engineering"  
};
```

In an object:

- Keys represent properties.
- Values can be strings, numbers, arrays, functions, or other objects.
- Properties can be accessed, added, updated, or deleted.
- Functions inside objects are called methods.

The training document specifically covers property access, adding/updating properties, deleting properties, methods, and built-in object methods.

10. DOM Manipulation

What is DOM?

DOM stands for **Document Object Model**.

It is a programming interface for HTML and XML documents. The DOM represents the webpage as a **tree of nodes**, including elements, attributes, and text. JavaScript can manipulate this structure dynamically.

Importance of DOM

DOM manipulation allows JavaScript to:

- Change webpage content.
- Update text and images.
- Modify styles.
- Create new elements.
- Respond to user interactions.
- Convert static webpages into dynamic webpages.

Example

```
document.getElementById("heading").textContent = "Hello!";
```

This changes the content of an HTML element dynamically.

11. Event Handling

An **event** is an action that occurs in a browser, such as clicking a button, pressing a keyboard key, or moving the mouse.

JavaScript can listen for these events and execute specific code in response.

Common Events

Event	Description
onclick	Occurs when an element is clicked
onmouseover	Occurs when the mouse moves over an element
onkeydown	Occurs when a key is pressed
onkeyup	Occurs when a key is released
onsubmit	Occurs when a form is submitted
onchange	Occurs when an input value changes

Example:

```
button.onclick = function() {  
    alert("Button clicked!");  
};
```

12. Callback Functions

A **callback** is a function passed as an argument to another function. It allows one function to call another function later.

Example:

```
function greet(name, callback) {  
    console.log("Hello " + name);  
    callback();  
}
```

```
function message() {  
    console.log("Welcome!");  
}
```

```
greet("Divya", message);
```

Uses of Callbacks

- **Reusability:** The callback can be changed without changing the main function.
- **Asynchronous handling:** Useful for operations such as API requests and file reading.
- **Control flow:** Defines what should happen after an operation is completed.

13. Promises

A **Promise** is a JavaScript object used to handle asynchronous operations.

A Promise represents a value that may become available immediately, later, or may never become available.

States of a Promise

1. **Pending** – Operation is still in progress.
2. **Fulfilled** – Operation completed successfully.
3. **Rejected** – Operation failed.

Why Promises Are Used

- Avoid callback hell.
- Provide cleaner asynchronous code.
- Allow `.then()` and `.catch()` for handling results and errors.
- Support chaining of multiple asynchronous operations.

Example:

```
let promise = new Promise((resolve, reject) => {  
    resolve("Success");  
});
```

promise

```
.then(result => console.log(result))  
.catch(error => console.log(error));
```

14. Async and Await

`async` and `await` provide a cleaner way of working with Promises. They make asynchronous code easier to read and understand.

async

The `async` keyword indicates that a function returns a Promise.

await

The `await` keyword waits for a Promise to be fulfilled or rejected before continuing execution of that `async` function.

Example:

```
async function getData() {  
    try {  
        let result = await promise;  
        console.log(result);  
    } catch (error) {
```

```
        console.log(error);
    }
}
```

Advantages

- Cleaner and more readable code.
- Avoids long `.then()` chains.
- Makes asynchronous operations easier to understand.
- Allows error handling using `try...catch`.

15. Closures

A **closure** is created when a function remembers and can access variables from its outer scope even after the outer function has finished executing.

Example:

```
function counter() {
    let count = 0;

    return function() {
        count++;
        return count;
    };
}
```

```
let increment = counter();
```

```
console.log(increment());
```

```
console.log(increment());
```

Here, the inner function remembers the count variable.

Uses of Closures

- **Data privacy:** Keeps variables hidden.
- **Encapsulation:** Allows controlled access to data.
- **Stateful functions:** Useful for counters and caching.
- **Event handlers:** Can preserve required data.
- **Functional programming:** Forms the basis of several advanced JavaScript patterns.